CO3-AT2-Case study

Name : Gayathri N P

Reg .No. 192421362

Title:

Case Study: Real-Time Online Gaming Leaderboard Using Quick Sort Algorithm

Description of the Real-World Problem

Online multiplayer games such as PUBG, Free Fire, Valorant, and Call of Duty maintain live leaderboards to display player rankings based on their scores. As thousands of players continuously gain points, the leaderboard must be updated quickly and efficiently.

The gaming server receives new scores every second, making it necessary to sort player scores rapidly so that the highest-scoring players appear at the top of the leaderboard with minimal delay.

Quick Sort is a suitable sorting algorithm because it uses the Divide and Conquer strategy, making it highly efficient for large datasets in average cases.

Problem Statement

Design an efficient sorting mechanism for an online gaming leaderboard where player rankings are updated in real time. The algorithm should sort player scores in descending order while minimizing execution time. Analyze the effect of pivot selection on Quick Sort's performance and suggest suitable optimizations to avoid worst-case scenarios.

Algorithm Used

Quick Sort Algorithm (Divide and Conquer)

Steps:

1. Select a pivot element.
2. Partition the array into two parts:
    - Elements greater than the pivot (for descending order)
    - Elements smaller than the pivot.

3. Recursively apply Quick Sort to both partitions.
4. Continue until each partition contains one or zero elements.
5. Combine all partitions to obtain the sorted leaderboard.

Justification for Choosing the Algorithm (Why This Algorithm?)

Quick Sort is chosen because:

- It has an average time complexity of **O(n log n)**.
- It is faster than many comparison-based sorting algorithms in practice.
- It efficiently handles large numbers of player scores.
- Divide and Conquer reduces the overall sorting workload.
- It requires only a small amount of additional memory.
- Randomized pivot selection significantly reduces the probability of worst-case performance.

Complexity Analysis

Case	Time Complexity
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**$O(n^2)$**
Space Complexity	**O(log n)**

Worst Case Occurs When

- Pivot is always the smallest element.
- Pivot is always the largest element.
- Input is already sorted or reverse sorted while choosing the first or last element as pivot.

Optimization

Using Randomized Pivot Selection or Median-of-Three Pivot greatly reduces the chance of worst-case performance.

Manual Execution (Step-by-Step Process)

Player Scores

[450, 620, 510, 700, 580]

Step 1

Choose pivot = **580**

Partition:

Left:

620	700

Right:

450	510

Array becomes

[620,700] 580 [450,510]

Step 2

Sort Left

620	700

pivot = 620

Result

700	620

Step 3

Sort Right

450	510

Pivot = 450

Result

510	450

Final Leaderboard

Rank	Score
1	700
2	620
3	580
4	510
5	450

The leaderboard is successfully sorted in descending order.

Applications

- Online gaming leaderboards
- Sports ranking systems
- Live auction systems
- Stock market ranking
- E-commerce product sorting
- University ranking systems
- Competition scoreboards

Advantages

- Very fast in average cases.
- Uses Divide and Conquer approach.
- Efficient for large datasets.
- Requires less additional memory.
- Average complexity is **O(n log n)**.
- Randomized pivot improves performance.

Limitations

- Worst-case complexity is **$O(n^2)$**.
- Performance depends heavily on pivot selection.
- Recursive implementation may cause stack overflow for extremely large datasets.
- Not a stable sorting algorithm.

Comparison with Other Algorithms

Algorithm	Best	Average	Worst	Stable	Suitable for Gaming Leaderboard
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	No	Excellent
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes	Good
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Poor
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Poor
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Suitable only for small datasets

Screenshot of the Program

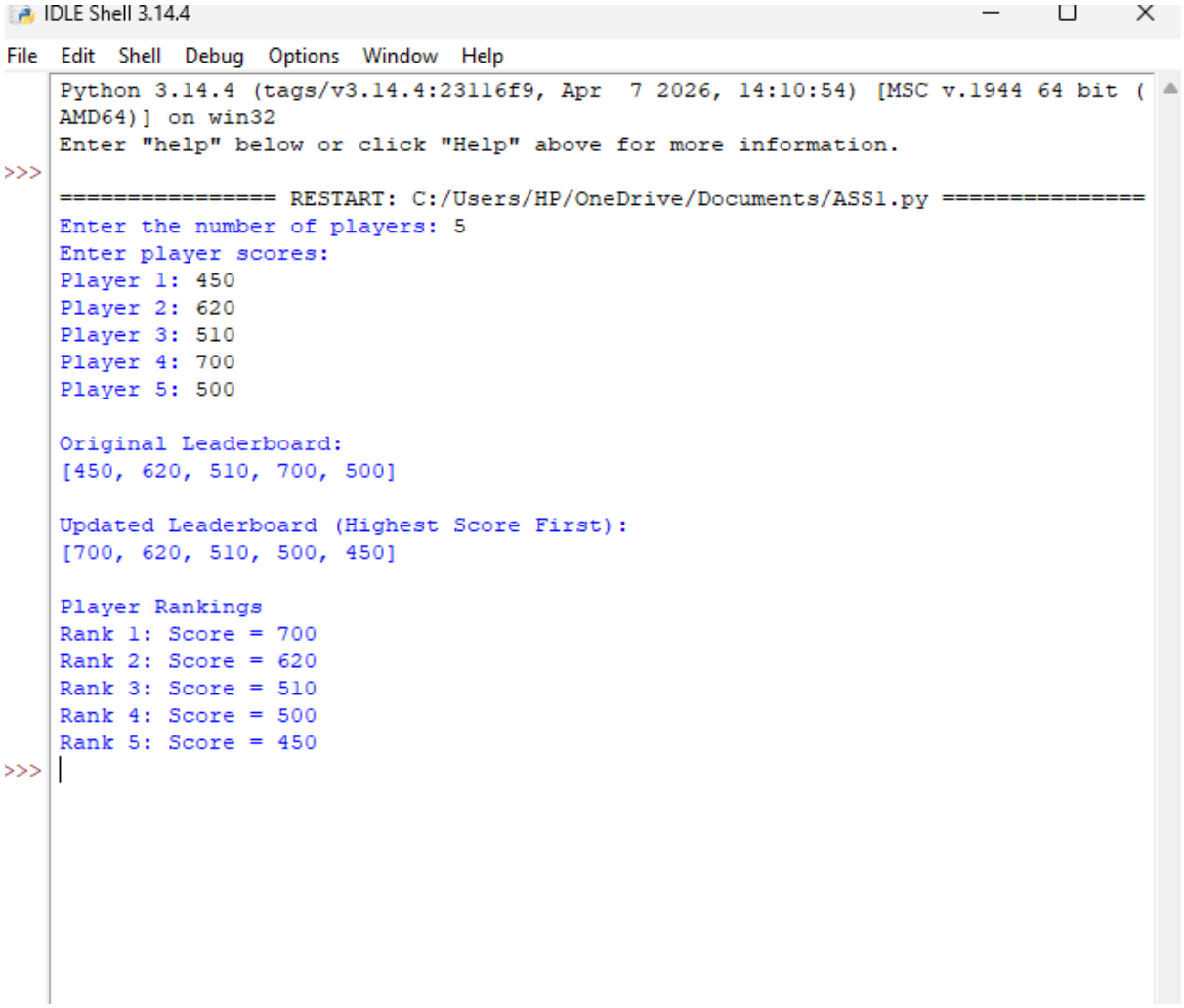
ASS1.py - C:/Users/HP/OneDrive/Documents/ASS1.py (3.14.4)

File Edit Format Run Options Window Help

```
import random
def partition(arr, low, high):
    # Select a random pivot
    pivot_index = random.randint(low, high)
    arr[pivot_index], arr[high] = arr[high], arr[pivot_index]
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        # Descending order
        if arr[j] >= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
# ---------------- Main Program ----------------
n = int(input("Enter the number of players: "))
scores = []
print("Enter player scores:")
for i in range(n):
    score = int(input(f"Player {i + 1}: "))
    scores.append(score)
print("\nOriginal Leaderboard:")
print(scores)
quick_sort(scores, 0, len(scores) - 1)
print("\nUpdated Leaderboard (Highest Score First):")
print(scores)
print("\nPlayer Rankings")
for i, score in enumerate(scores, start=1):
    print(f"Rank {i}: Score = {score}")
```

Output Screenshot



Conclusion

Quick Sort is an effective algorithm for maintaining real-time online gaming leaderboards because of its high average-case performance of **O(n log n)** and efficient Divide and Conquer strategy. However, poor pivot selection can lead to the worst-case time complexity of **$O(n^2)$**. This limitation can be minimized by using **Randomized Pivot Selection** or the **Median-of-Three** method, ensuring consistent performance even for large and dynamically changing player datasets. Therefore, Quick Sort with randomized pivot selection is an excellent choice for real-time leaderboard management in online gaming systems.